Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

ShrinkIt : Smart File Compression and Management System

Team Name:	*ShrinkIt*
Team member 1 (Team Lead)	*Gupta,Tammana-2510370576* *tamannagpt13@gmail.com* 
Team member 2	*Agarwal,Gaurika-2510011412* *gaurikaagarwal.24@gmail.com* 
Team member 3	*Joshi,Komal - 2510012297* *komaljoshi7667@gmail.com* 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

The primary objective of ShrinkIt is to create an efficient file compression and management system that assists users in minimizing storage space, ensuring the preservation of original data without any loss.The system employs lossless compressions, primarily utilizing the Huffman Coding algorithm, to efficiently compress files and ensure accurate decompression.In addition to basic compression and decompression, ShrinkIt emphasizes offering helpful compression analysis to help users in understanding the efficiency of compression.The system will show details such as original file size, compressed file size, compression ratio, space saved percentage, and the time it takes to compress or decompress.This allows users to compare and evaluate the efficiency of file compression.

Another crucial objective of ShrinkIt is detecting duplicate files.The system also detects duplicate files by comparing their relevant files details,which helps in finding out unnecessary duplicates to minimize the excess storage.

The project uses concepts like Data Structures and Algorithms, C++ Object-Oriented Programming and File Handling to develop a practical solution for achieving effective storage management.Overall ShrinkIts aim is to combine file compression, analysis,detection of duplicates and file management as a single system.

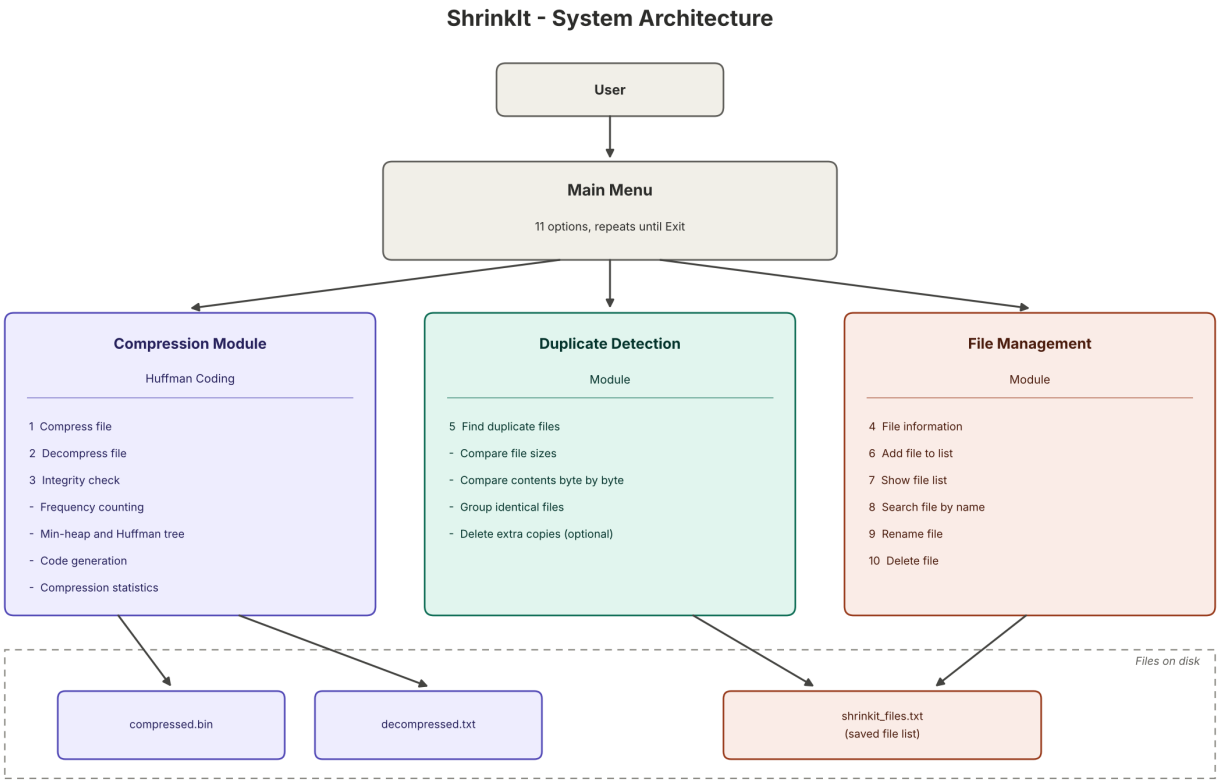
Updated Project Approach and Architecture

ShrinkIt is a command line application written in C++. The program is designed to shrink text files and allow the original content to be restored when needed. The application will be built using modules, each responsible for a specific task. This structure allows every team member to work on a part and the modules will be combined later to form a complete working project.

First the input text file will be analyzed to determine how often each character appears. After collecting the character frequencies a Huffman tree will be constructed. The binary codes assigned to each character will be unique based on the structure of this tree. These codes will then be used to encode the file and create a version. The compressed file can later be decoded to restore the original content without any data loss.

To make the program work correctly we will use C++ libraries. We will use iostream for input and output operations fstream for reading and writing files and a queue data structure to manage the priority queue needed during Huffman encoding.

The program will use data structures such as trees to build the Huffman tree. We will also apply hashing, searching and sorting techniques where needed.Since ShrinkIt is meant to be a command-line tool the user will interact with it by typing commands and viewing results directly in the console window. There is no need, for databases or complex communication protocols. The project focuses on file operations. Each module can be tested independently to make sure it works as expected. Once all parts are working correctly they will be integrated into a command-line utility to verify that the entire system functions as intended.



Tasks Completed

Task Completed	Team Member
1-The project's objectives ,characteristics and responsibilities were finalised.	All Members
2-Requirement analysis, system design and File handling module for reading and writing text and binary files.	All Members
3-Implemented Huffman Coding using frequency counting, min-heap, and building Huffman tree.	Tamanna Gupta
4-Developed Compression and decompression with compression statistics like size, ratio, space saved, time taken.	Komal Joshi
5-Added Integrity check to verify the decompressed file matches the original	Gaurika Agarwal
6-Built Duplicate file detection using size and content comparison	Tamanna Gupta
7-Created File management features and integration of all OOP-based modules into one application	All Members

Challenges/Roadblocks

A significant problem was the storage of the Huffman tree for decompression, which was solved by storing the character frequency table in the file header to reconstruct the tree.
Since C++ uses bytes for storage, while Huffman coding uses variable-length bits, the variable-length bits were packed with bitwise operations and stored with the number of bits that need to be ignored. Compression of already compressed files such as JPG, MP3, and ZIP showed insignificant compression results; therefore, the statistics will show the space savings, and a warning will be added for such files.
Large files were slow and memory-intensive; therefore, the files were read in blocks.
Furthermore, finding duplicates by name or size was unreliable; thus, the size needs to be checked first, followed by the contents or hash. Finally, merging of different modules was complicated by the differences in the code, which were solved by using a common class, coordination with each member, and testing each module before merging.

Tasks Pending

Task Pending	**Team Member (to complete the task)**
1-Replace Huffman coding with a new compression method - RLE and LZ	Komal,Gaurika,Tamanna
2-Compare the new method with Huffman coding	Komal,Gaurika,Tamanna
3-Debug and optimize the code	Komal,Gaurika,Tamanna

Project Outcome/Deliverables

ShrinkIt provides a complete C++ storage management system that integrates file compression, performance evaluation, duplicate identification, and file management in one application.
1-Lossless compression and decompression:Utilizing Huffman Coding, files are compressed to conserve storage and space while ensuring that no data is lost during the process, as they are then decompressed and restored to their original state.
2-Compression analysis report: For each operation, the system shows the initial file size, the reduced file size, the compression ratio, the percentage of space saved, and the duration to compress or decompress, allowing users to assess efficiency.
3-Duplicate file detection: The system checks file details to identify duplicates, allowing users to find and remove extra copies, thus freeing up storage space.
4-Integrated file management:For easy file organization and handling, integrated file management features allow users to work efficiently in one location.
5-Practical use of core concepts:Applying Data Structures, C++ Object-Oriented Programming, and File Handling, the project aims to solve an real storage issue.

Progress Overview

The current status of the ShrinkIt project is that about 75percent of the work is done. The planning, requirement analysis, and system design activities have been completed. The Huffman Coding module, file handling, file management features, compression statistics, integrity check, and duplicate file detection have been completed and incorporated into an application.

Behind schedule: Our own compression method using RLE and an LZ sliding -window coder is not implemented in the project yet , while Huffman Coding will eventually be replaced with it. Besides, the compression and decompression process of new data compression method is also to be designed

Ahead of schedule: Duplicate detection of files was completed. The design of the new compression method has already been finished,giving clear plan for next phase.

Codebase Information

Repository Link : https://github.com/tamannagpt07/Shrink-It-Smart-File-Compression-and-Management-System.git
Branch : main
The repository consists of the C++ code of the ShrinkIt project. The program allows users to compress and decompress files, check the integrity, view compression statistics, find duplicates, and manage their files. In the C++ part, we have implemented Huffman Coding using min-heap, compressed and decompressed files, checked the files' integrity comparing to the original one, found duplicates comparing their size and content, and managed the list of the saved files. We have separate commits for every feature; it is easier to see what changes have been applied. We will update the code on GitHub, fix errors, test every part, and develop our own compression method using RLE and LZ to compare them with Huffman Coding and make the project as one system.

Testing and Validation Status

Test Type	**Status (Pass/Fail)**	**Notes**
Compression and decompression	Pass	Shrinks a file to save storage space,restores it to its exact original form without any data loss.
Integrity check	Pass	Confirms that a decompressed file is exactly the same as the original
Duplicate file detection	Pass	Finds files with identical content by comparing file size and then contents.
RLE and LZ method	Pending	Will be compared against Huffman Coding.
OOP and DSA testing	Pass	Checks classes and modules function ,ensure priority queue, Huffman tree, and file comparison.

Deliverables Progress

Completed -

1-Lossless compression and decompression using Huffman Coding.
2-Compression statistics (size, ratio, space saved, time taken).
3-Integrity check.
4-Duplicate file detection.
5-File management features.

In Progress-

1-Debugging and optimization.

Pending-

1-Own compression method using RLE and LZ
2-Comparison of the new method with Huffman Coding